UNIVERSITÉ DE LORRAINE

Travaux pratiques d'Informatique Industrielle

Portail de licence ISFATES et SPI

Olivier HABERT
David CHAPRON
Stefan BITTNER

V7, 05/03/2026

Résumé

Ces travaux pratiques ont pour objectif de vous faire découvrir l'informatique industrielle, art d'interfacer l'électronique et l'informatique pour piloter des systèmes industriels. Nous utiliserons comme partie opérative des robots pédagogiques permettant une programmation graphique ou une programmation en langage évolué *(langage C sous environnement de développement Arduino)*. Ces TP vont également vous initier aux grands principes de l'algorithmique pour le développement d'applications visuelles, sonores, de pilotage de la mobilité d'un robot, … en utilisant des capteurs extéroceptifs ou proprioceptifs.

Nous allons tout d'abord utiliser un environnement de programmation graphique souvent utilisé pour découvrir la programmation chez les adolescents puis glisser vers un environnement de développement plus professionnel basé sur un langage de programmation évolué, C.

En espérant que cette approche de l'informatique vous donnera envie de poursuivre dans les formations en sciences pour l'ingénieur en électronique, énergie électrique, automatique, informatique industrielle, instrumentation, capteurs, systèmes embarqués, robotique, internet des objets, …

Contenu

RESUME 2

PRESENTATION DE L'INFORMATIQUE INDUSTRIELLE 5

Définitions 5

Compétences de l'informatique industrielle 6

Composants utilisés en informatique industrielle 6

LE MATERIEL DES TRAVAUX PRATIQUES 12

Le robot MBOT RANGER 12

La carte Auriga 12

Le robot STARTER BLUETOOTH 14

LE LOGICIEL MBLOCK, PROGRAMMATION GRAPHIQUE 18

Présentation 18

Mise en route 18

Premier programme 19

Les boucles 20

Les variables 20

Le capteur de son 21

Les instructions conditionnelles 21

Le capteur ultrason 22

L'accéléromètre / gyroscope 22

Les blocs pour la mobilité 23

PROGRAMMATION AVANCEE SOUS L'ENVIRONNEMENT ARDUINO 27

L'environnement développement Arduino 27

Structure d'un programme Arduino 30

Connexion avec la carte **31**

Eléments de langage **31**
Boucle simple 32
Moniteur série 32
Instructions conditionnelles 33
Opérateurs booléens et comparaisons 33
Syntaxe générale 34
Variables 34

Mon premier programme avec le robot **35**

Les capteurs de lumière **35**

Le capteur à ultrasons **36**

Le capteur de son **36**

Le gyroscope **37**

Pilotage du robot **37**

ANNEXES **40**

Gyroscope **40**

IDE Arduino : installation des pilotes **41**

Liste des exercices **42**

Présentation de l'informatique industrielle

Définitions

***Définition Web :** Ce domaine de la technologie est une branche de l'informatique appliquée qui couvre l'ensemble des techniques de conception et de programmation, de systèmes informatisés à vocation industrielle, qui ne sont pas des ordinateurs : Automate électronique, Machine numérique, Outil, Instrument intelligent.
Les appareils concernés contiennent au moins un microprocesseur ou microcontrôleur, ainsi que des coupleurs, (tampons d'entrées-sorties).

***Autre définition :** Art d'associer l'électronique numérique et analogique, l'informatique, les moyens de communication et la physique appliquée pour :

- Mesurer (capteurs) et acquérir (systèmes d'acquisition + logiciels),

- Agir (actionneurs) localement ou à distance sur l'actionneur,

- Transporter l'information mesurée (par réseau de terrain, internet, …),

- Superviser un processus industriel (localement ou à distance),

- Archiver l'information, mettre en forme l'information,

- Appliquer une démarche qualité pour maintenir ou obtenir une certification ISO,

Compétences de l'informatique industrielle

Électronique

Analogique :

Électronique instrumentale, de puissance, d'amplification, de filtrage

Numérique :

Systèmes micro programmés, interfaçage, gestion de périphériques, bus de terrain …

Informatique

Analyse fonctionnelle d'un problème, algorithmique, programmation, maîtrise des possibilités d'un système d'exploitation, communication, archivage de données, … conception de systèmes automatisés, réseaux d'automates, techniques d'asservissement numérique, …

Physique appliquée

Choix des capteurs adaptés, interfaçage électrique des capteurs, maîtrise des conditions expérimentales de mesure, bonnes connaissances des phénomènes physique mise en jeu, précision de la mesure, validité de la mesure, …

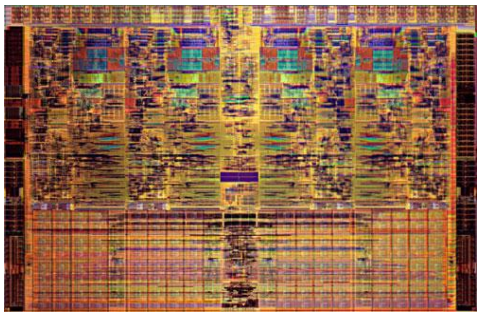
Composants utilisés en informatique industrielle

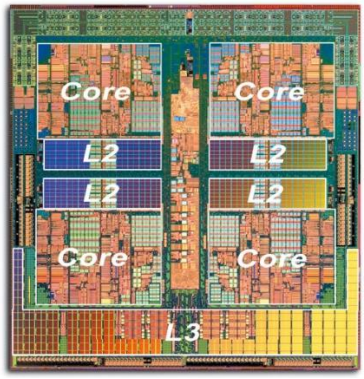
Les microcontrôleurs

Il correspond au cerveau du système d'informatique industrielle. C'est lui qui va traiter les informations provenant des capteurs et qui va donner la réponse voulue aux actionneurs. On peut considérer qu'il est constitué de 4 parties :

Le microprocesseur : Il va prendre en charge la partie traitement des informations et envoyer des ordres. Il est lui-même composé d'une unité arithmétique et logique (UAL) et d'un bus de données. C'est donc lui qui va exécuter le programme embarqué dans le microcontrôleur.

Un microprocesseur est composé de milliards de transistors qui fonctionnent en commutation (https://www.01net.com/actualites/30-milliards-de-transistors-sur-une-puce-de-la-taille-d-un-ongle-le-nouveau-record-d-ibm-1179677.html).





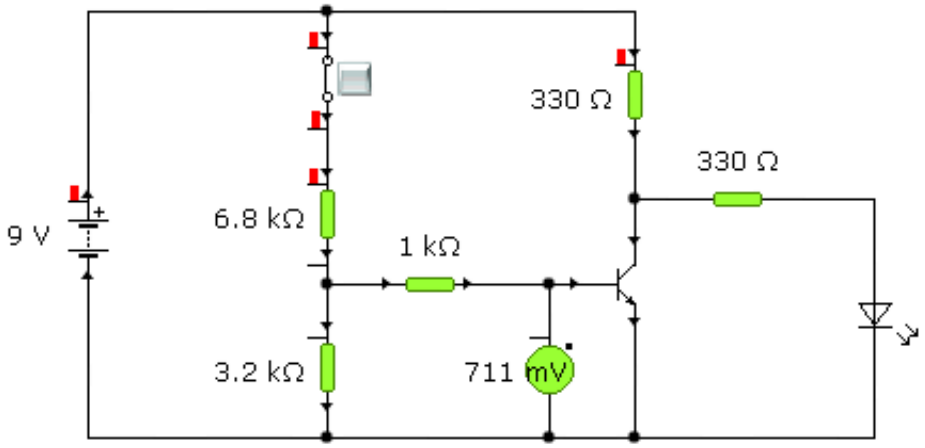
Une <u>mémoire de données</u> ou mémoire vive (RAM) dans laquelle seront entreposées les données temporaires nécessaires aux calculs. C'est en fait la mémoire de travail qui est donc volatile : elle est effacée sans alimentation

Une <u>mémoire programmable</u> (EEPROM), qui va contenir les instructions du programme pilotant l'application à laquelle le microcontrôleur est dédié. Il s'agit ici d'une mémoire non volatile puisque son contenu est préservé même sans alimentation.

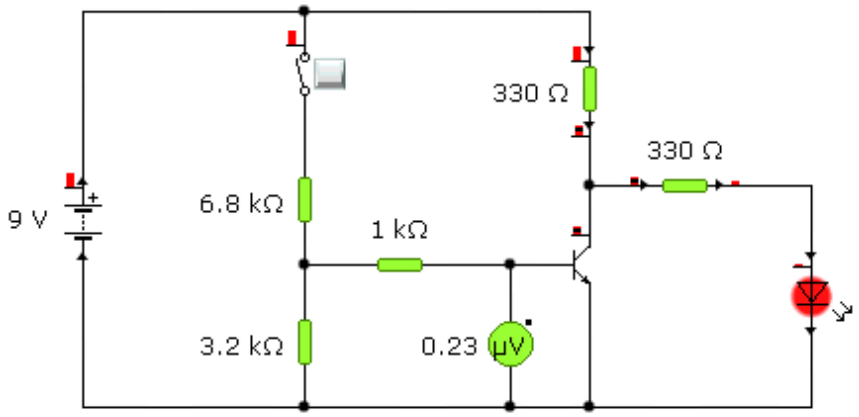
La dernière partie correspond aux <u>ressources auxiliaires</u>. Celles-ci sont généralement :

- Ports d'entrées / sorties parallèle et série, protocoles d'échange de données I2C, SPI, CAN, … avec un très faible nombre de fils,
- Des timers pour générer ou mesurer des signaux avec une grande précision temporelle, pour générer des interruptions, …
- Des convertisseurs A/N ou N/A pour traiter les signaux analogiques,
- Une connectivité réseau par Wifi ou BlueTooth.

Principe de fonctionnement d'un transistor en commutation :

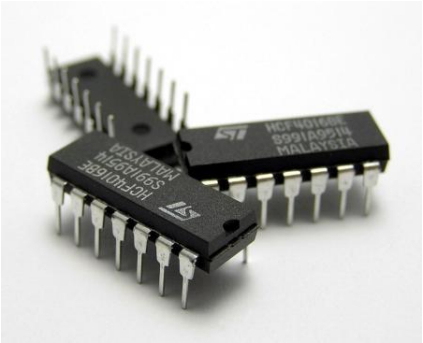


Transistor passant (LED s'éteint)



Transistor bloqué (LED s'allume)

Les circuits logiques de base de l'informatique industrielle



Portes ET, OU, NON, OU EXCLUSIF, NON ET, NON OU, NON OU EXCLUSIF, MULTIPLEXEURS, DECODEURS, TRANSCODEUR, ADDITIONNEURS, BASCULES RS, BASCULES JK, COMPTEURS, REGISTRES A DECALAGE, …

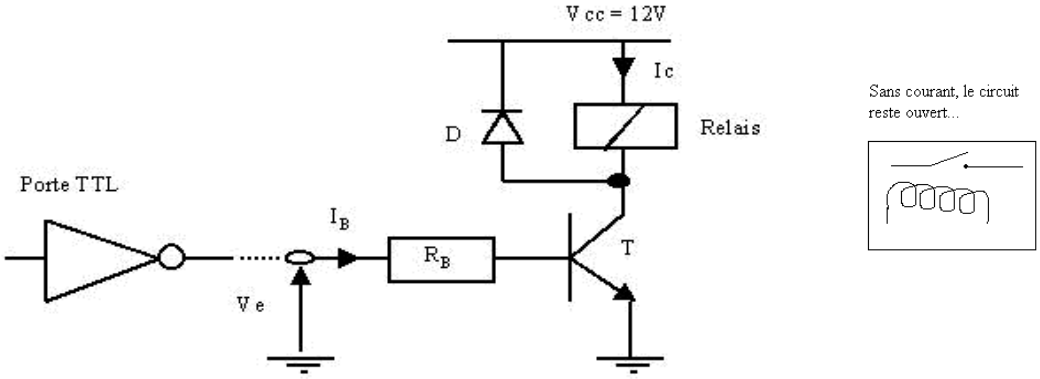
FONCTION	EQUATION	SYMBOLES			TABLES DE VERITE
		International	Français	Allemand	
NON	$S = \overline{a}$	a S	1		a S 0 1 1 0
ET	$S = a . b$	a b S	&		a b S 0 0 0 0 1 0 1 0 0 1 1 1
NAND	$S = \overline{a . b}$	a b S	&		a b S 0 0 1 0 1 1 1 0 1 1 1 0
OU	$S = a + b$	a b S	≥ 1		a b S 0 0 0 0 1 1 1 0 1 1 1 1
NOR	$S = \overline{a + b}$	a b S	≥ 1		a b S 0 0 1 0 1 0 1 0 0 1 1 0
OU Exclusif	$S = a \oplus b$	a b S	= 1	e	a b S 0 0 0 0 1 1 1 0 1 1 1 0
NOR Exclusif	$S = \overline{a \oplus b}$	a b S	= 1	e	a b S 0 0 1 0 1 0 1 0 0 1 1 1

Fig. 62. - Symboles graphiques des différentes fonctions logiques.

Les composants pour l'interfaçage Informatique ←→Electronique

Les circuits d'interfaçage permettent d'isoler électriquement la partie informatique de la partie électronique.

Les relais électromécaniques ou analogiques :

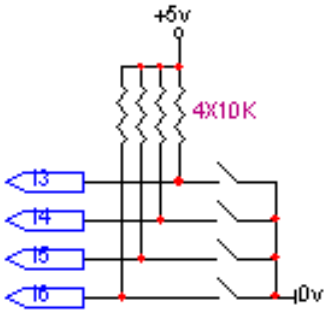


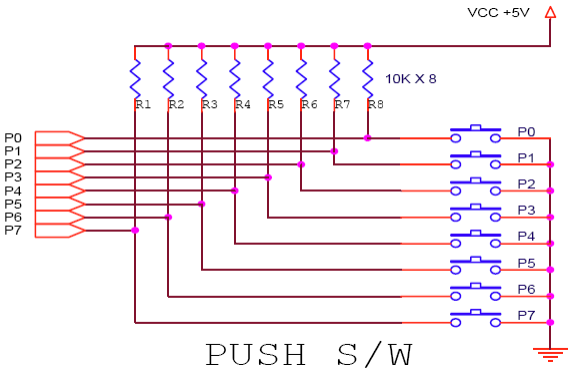
Les optocoupleurs

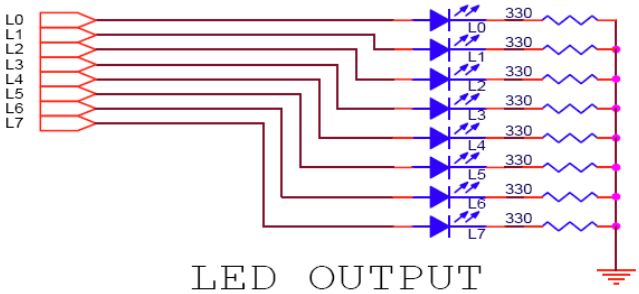
Principe : Les optocoupleurs sont constitués de deux composants intégrés dans un même boîtier : une DEL et un Phototransistor...

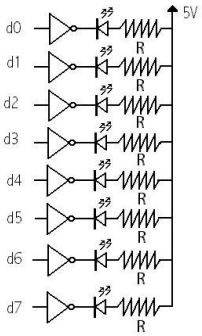
Les entrées-sorties

Exemples de montages de base pour bascule (aussi dit « flip-flop » : élément qui peut préserver un état), boutons poussoirs, LED









Les 8 tampons inverseurs (74LS04) permettent :

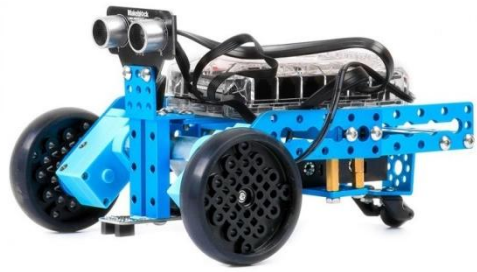
- de garantir une alimentation en courant, des LEDs, pour un éclairage correct
- de rendre la logique positive (à la suite du circuit précédent des Push S/W qui est câblé en logique négative)

Le matériel des travaux pratiques

Nous allons utiliser des robots pédagogiques très connus dans les lycées voire collèges pour découvrir la programmation. Ces robots appartiennent à la famille MBOT du constructeur MAKEBLOCK. Il existe deux versions :

Le robot MBOT RANGER

Il est vendu avec la possibilité de l'assembler de trois manières différentes (char, voiture à 3 roues, voiture auto-équilibré). Nous l'utiliserons dans la version char. Ce robot est pré-équipé avec quelques capteurs, LED, afficheurs, drivers de moteurs à courant continu.



Il est basé sur une carte compatible ARDUINO que nous allons décrire.

La carte Auriga

Dotée d'un processeur ATmega 2560 compatible avec l'IDE Arduino, elle peut prendre en charge 2 moteurs à courant continu de 6 à 12 V. Elle possède 10 ports dont 4 pouvant piloter des moteurs à l'aide d'un module "Me Dual DC Motor Driver". La carte AURIGA intègre différents capteurs (lumière, son, température, et panneau LED RGB). Il est possible d'y ajouter un module Bluetooth ou 2,4 GHz grâce à un port dédié.

Elle comporte exactement (voir la figure suivante pour les emplacements sur la carte) :

- Une interface de puissance pilotant les moteurs à courant continu **avec encodeur** sous 6 à 12 V et 2,4 A max,
- Deux capteurs de lumière,
- Un capteur de bruit,
- Un capteur de température,
- Un gyroscope,
- Un buzzer,

- Un anneau de 12 LEDs adressables.

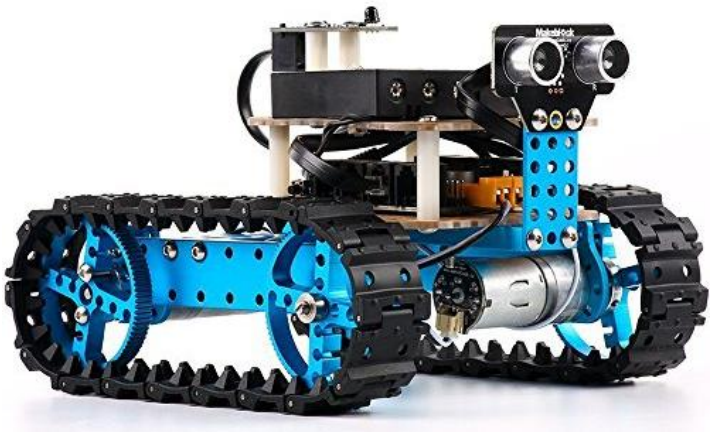
La carte est également équipée de connecteurs au format RJ25 permettant de connecter des modules additionnels à la norme MAKEBLOCK *(capteurs ultrasons, infrarouge, …)*. La connectique RJ25 suit un code couleur en fonction du type de port disponible sur chaque connecteur. Ils sont numérotés de 1 à 10, numéros souvent utilisés dans les programmes pour identifier les ports concernés. Les capteurs intégrés à la carte, donc hors connecteurs, sont associés également à des ports, voir la liste ci-dessous :

```
Port 0 { NC, NC } Not connected,

Port 1 { 5, 4 } red,

Port 2 { 3, 2 } red,

Port 3 { 7, 6 } red,

Port 4 { 9, 8 } red,

Port 5 { 16, 17 } grey,

Port 6 { A10, A15 } universal,

Port 7 { A9, A14 } universal,

Port 8 { A8, A13 } universal,

Port 9 { A7, A12 } universal,

Port 10 { A6, A11 } universal,

Port 11 { NC, A2 } light sensor 1,

Port 12 { NC, A3 } light sensor 2,

Port 13 { NC, A0 } temperature sensor,

Port 14 { NC, A1 } sound sensor.
```

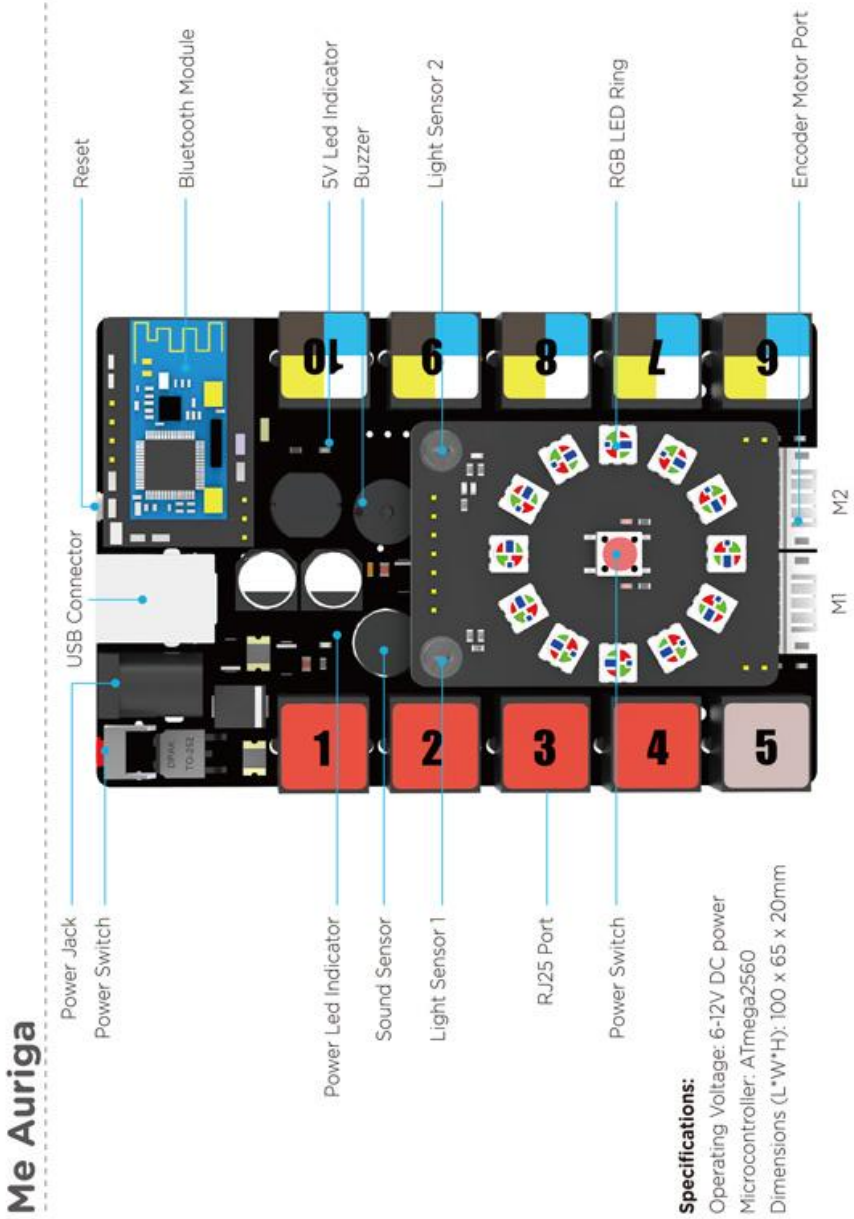
Le robot STARTER BLUETOOTH

Il repose sur une carte ORION compatible Arduino™ UNO-328 (base ATmega328) associé à un contrôleur de moteur spécialisé de type "TB6612", lequel sera directement capable de piloter 2 moteurs de courant continue (6 à 12 V / 1 A max.)

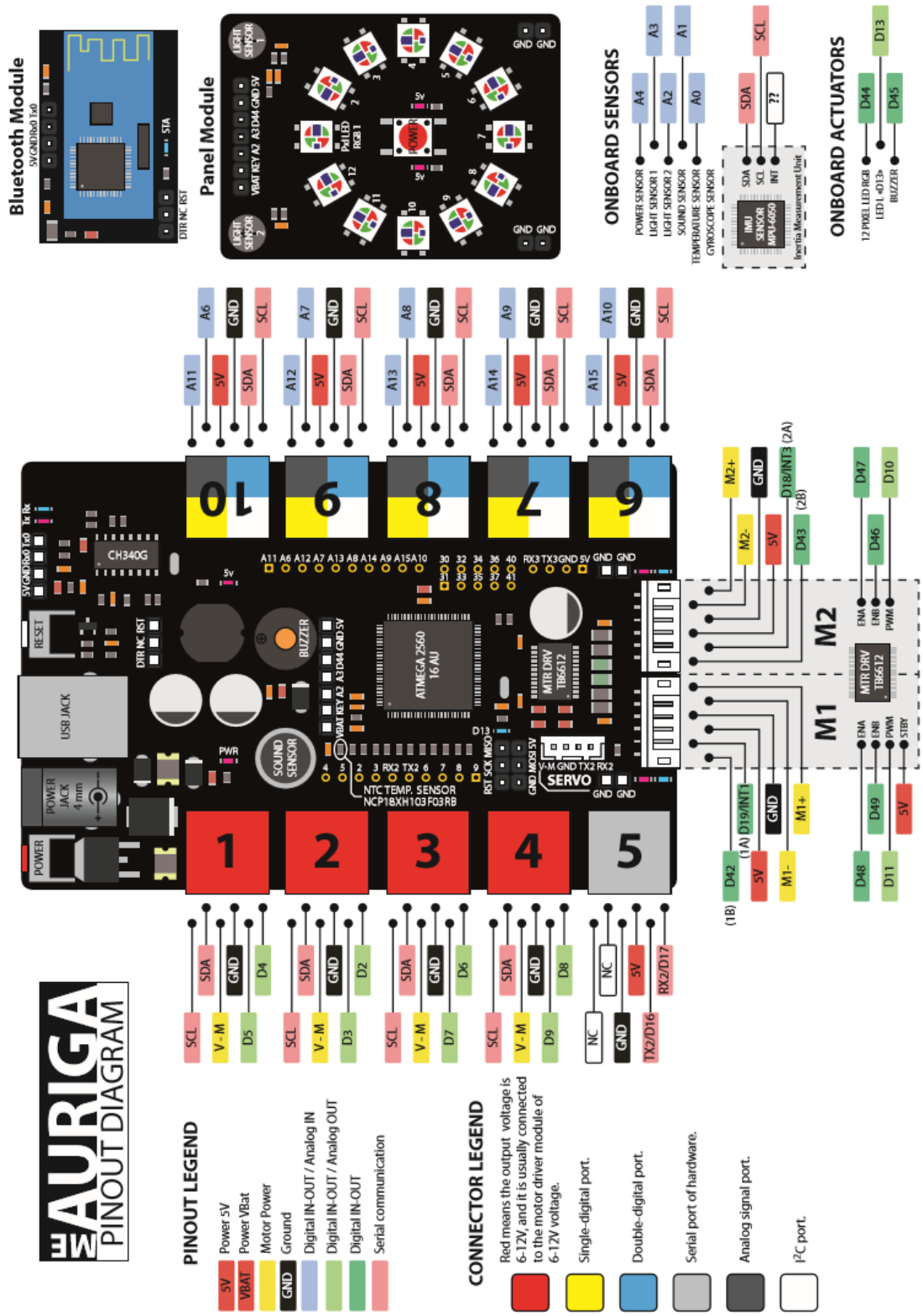
Ce robot n’est pas disponible pour les TPs.



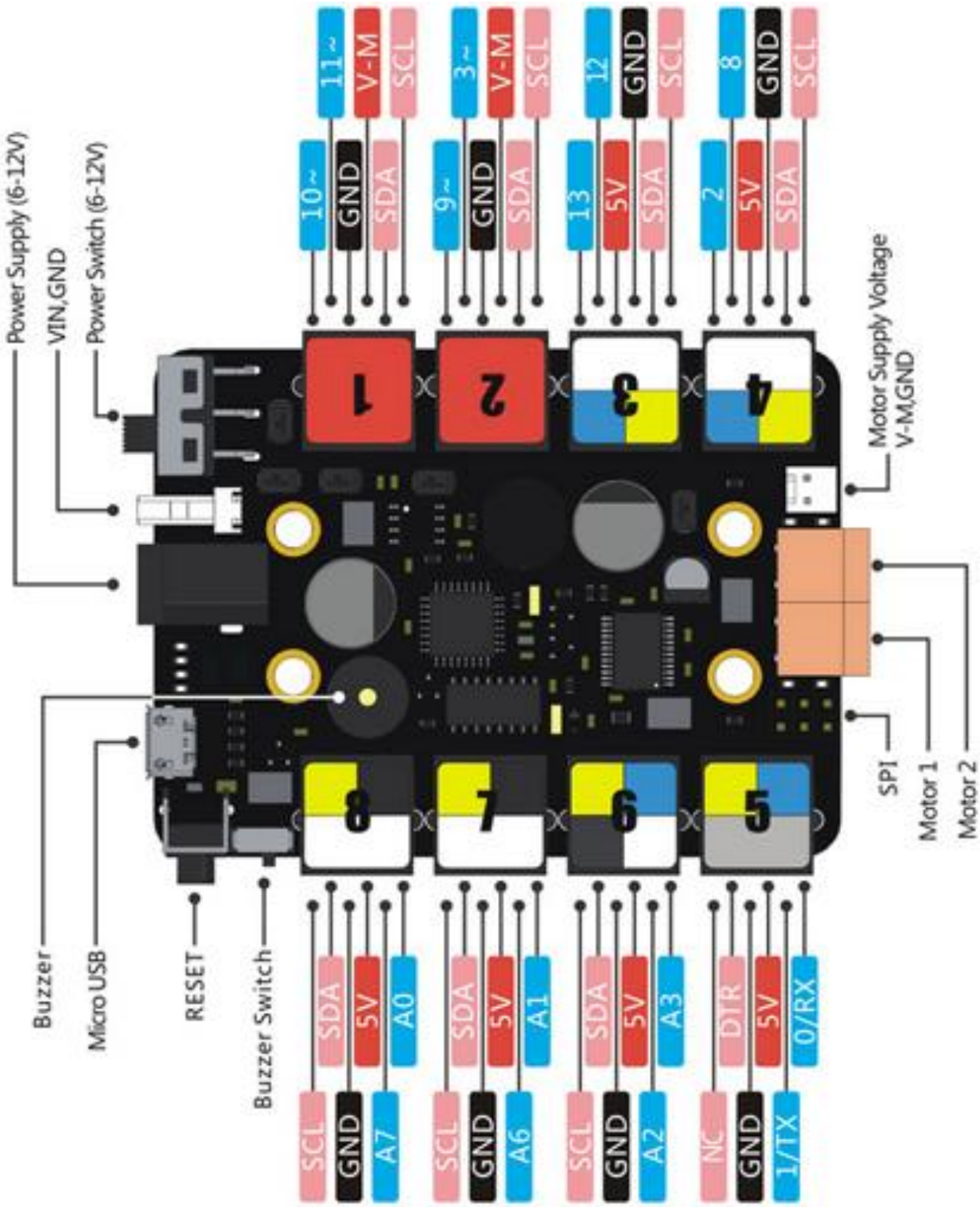
Carte ME Auriga pour MBOT Ranger



Carte ME Auriga pour MBOT Ranger



Carte Orion pour Starter Bluetooth



Le logiciel Mblock, programmation graphique

Présentation

Mblock est un logiciel gratuit permettant la programmation via une interface graphique. Ce logiciel est basé sur la l'environnement Open Source Scratch 2.0, faisant de lui le logiciel parfait pour l'apprentissage de la programmation sous le langage Arduino. Il est destiné à l'apprentissage de la programmation aux plus jeunes mais peut servir de base à tous pour apprendre les principes de la programmation avant d'aborder ceux-ci de manière plus professionnelle avec un environnement de programmation utilisant un langage évolué.

La version de Mblock que nous utilisons est la version 3.4.12. Elle est déjà installée sur les ordinateurs, et vous la trouvez également sur Arche.

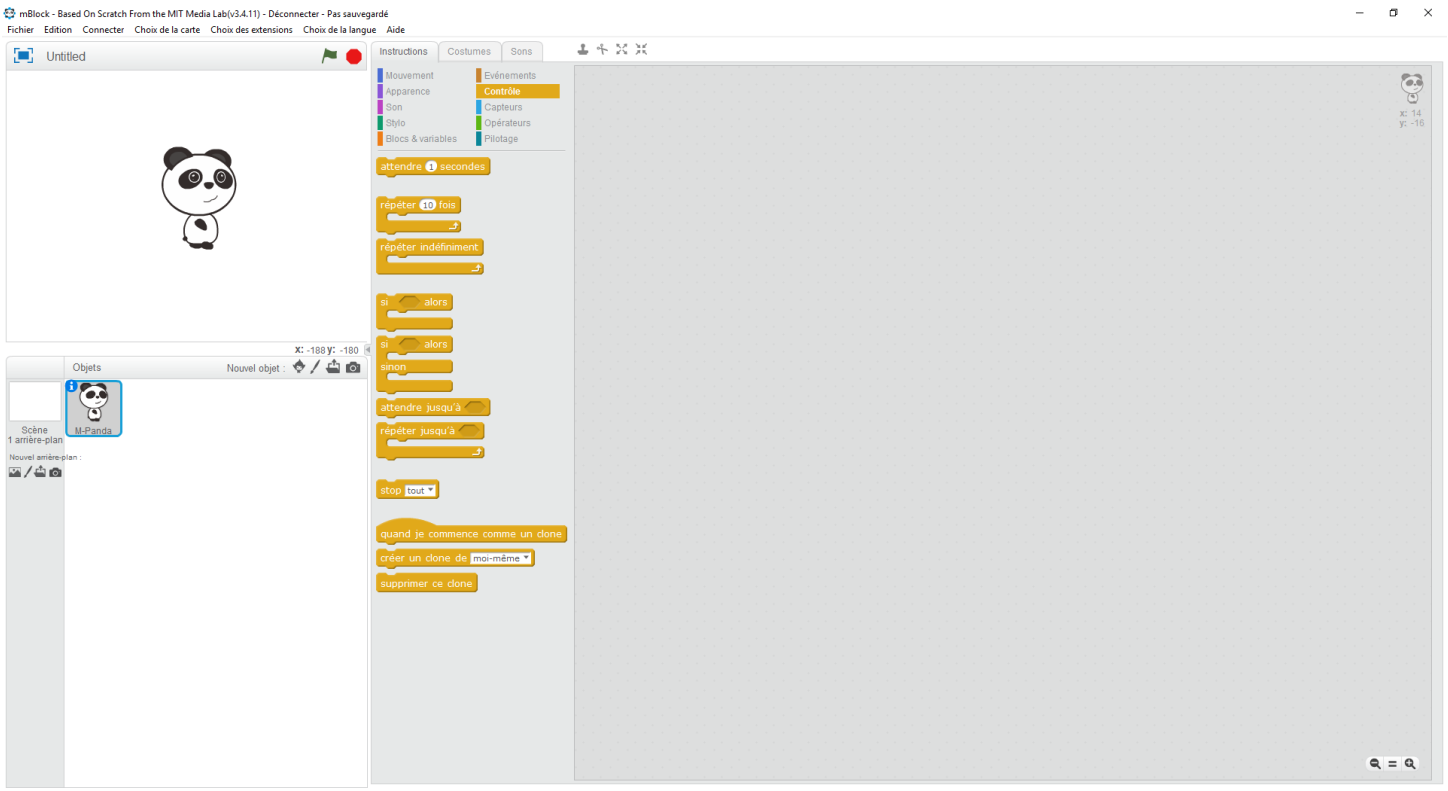
Mise en route

Lancer le logiciel et choisir la langue. Il va falloir maintenant paramétrer le logiciel pour effectuer la liaison avec la carte du robot. Dans les explications, nous travaillons avec un MBOT RANGER doté d'une carte AURIGA mais la démarche est équivalente avec un autre robot MakeBlock équipé d'une autre carte *(ORION par exemple)*.

- Aller dans le menu ***« Choix des extensions »*** et cocher ***« Makeblock »***,
- Connecter le robot avec le câble USB,
- Dans ***« Choix de la carte »,*** sélectionner Mbot Ranger (Auriga),
- Dans ***« Connecter »***, choisir ***« Par port série (COM) »*** et choisir un port. Vous pouvez trouver le numéro en déconnectant le câble USB,
- Pour tester votre choix, lancer ***« Réinitialiser le programme par défaut »*** dans le menu ***« Connecter »***, si ça bloque au niveau du téléversement, changer de port,
- Un BEEP retentit généralement lorsque la connexion est établie avec le robot,
- Dans un premier temps, nous allons rester en mode contrôle direct du robot : dans ***« Edition »***, le mode Arduino ne doit pas être coché.

L'interface se présente comme sur la figure suivante :

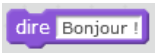
- Une zone à droite qui contiendra le programme,
- Une zone en haut à gauche ou on pourra effectuer des affichages via un avatar,
- Une zone en bas à gauche où on peut voir les actions sur l'avatar *(mouvement, apparence, affichage des variables …)*,
- Une boite de dialogue au centre avec un onglet « Instructions » affichant plusieurs catégories *(Mouvement, Apparence, …, Evénements, …)*,
- Les différents éléments de chaque catégorie s'affichent en dessous.



Premier programme

<u>Exercice 1 : capteur de luminosité</u> En mode direct *(commande du robot par le logiciel Mblock)*, glisser-déposer dans la zone de programme le bloc

de la catégorie événement. Ce bloc permettra de lancer le programme en cliquant sur le drapeau vert en haut à droite de la zone d'affichage. Nous allons maintenant programmer une action.

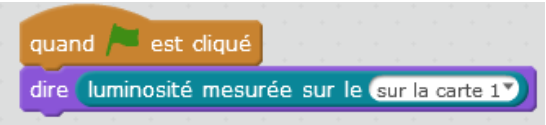
Placer le bloc 

de la catégorie Apparence en dessous du bloc précédent en l'encastrant à l'aide de l'encoche.

Superposer le bloc

de la catégorie Pilotage sur la zone de texte du bloc précédent (ou Bonjour ! apparait par défaut).

Voici ce que vous devez obtenir :

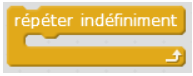


Cliquer alors sur le drapeau vert de la zone d'affichage et observer l'avatar de la zone d'affichage.

Modifier l'éclairement du capteur de luminosité *(Voir figure précédente pour localiser le capteur de lumière n° 1)* avec la main et relancer le programme pour chaque essai.

Questions pour le rapport : Quel est le sens de variation, amplitude de variation estimée, de la valeur affichée en fonction de l'éclairement ? Quelle est la valeur minimale et maximale ? Quel type d'entrée est utilisé pour ce capteur ? Expliquer la valeur maximale avec le type de composant électronique utilisé pour cette entrée ?

Les boucles

Exercice 2 : boucle infinie Dans un programme, il doit être possible de répéter un ensemble d'instructions de plusieurs fois à l'infini. Dans la catégorie Contrôle, récupérer le bloc 

. Détacher le bloc **Dire** et placer celui-ci à la place, remettre le bloc **Dire** à l'intérieur du nouveau bloc.

Question pour le rapport : En quoi diffère le fonctionnement du programme par rapport à précédemment ?

Exercice 3 : boucle N fois En utilisant les bloc

et

, faire 12 mesures espacées de 2 secondes.

Les variables

Les variables permettent de stocker des nombres, des caractères alphanumériques et sont indispensables au fonctionnement d'un programme pour stocker par exemple les opérandes et les résultats d'un calcul. Une variable se caractérise par un nom et par son type *(en programmation avancée)*. Il est possible de définir des variables sous Mblock :

- Aller dans la catégorie **Blocs & variables**,
- Cliquer sur ***« Créer une variable »***,
- Dans la boite de dialogue, préciser un nom court *(**i** par exemple)* ; n'utiliser pas des espaces ou des charactères spéciaux dans le nom (un _ est possible),
- Laisser coché ***« Pour tous les objets »*** et cliquer sur OK,
- La variable apparait normalement en dessous du bouton ***« Créer une variable »***, vous pouvez glisser-déposer cette variable dans la zone de programme,
- Avec le bloc

, vous pouvez incrémenter cette variable dans un programme.

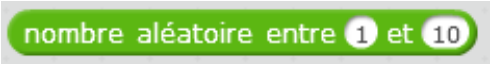
Exercice 4 : variable Créer une variable ***i***, l'afficher et l'incrémenter de 1 de manière infinie.

Exercice 5 : LEDs Sur le robot, vous avez un anneau avec 12 LED couleurs. Chaque LED peut se piloter *[allumée ou éteinte et la couleur en RVB (RGB en Anglais)]* par le bloc

.

Le premier paramètre est le numéro de la LED *(entre 1 et 12)* et les trois suivants sont les composantes Rouge, Verte et Bleu pour définir la couleur.

- Essayer le bloc pour allumer l'une des LED avec la couleur de votre choix,

- Allumer successivement une des 12 LED avec la couleur de votre choix après un délai d'une seconde. Utiliser une variable ***a*** qui est incrémentée pour adresser les LEDs et afficher cette variable dans l'avatar. Après la dernière LED on attendra 5 secondes avant d'éteindre toutes les LED.
- Définir 3 autres variables ***r, v, b***,
- A l'aide du bloc 

, affecter à chaque LED une couleur aléatoire.

<u>Question pour le rapport :</u> Renseignez-vous sur la synthèse additive des couleurs. Comment pouvez-vous générer les couleurs cyan, jaune, magenta et orange avec les LEDs ? Testez les différents mélanges des couleurs avec votre robot.

Le capteur de son

La carte présente un capteur de son, et le volume de son peut être déterminé par le bloc

.

<u>Exercice 6 : capteur de son</u> Créer un programme qui affiche le volume de son en continue.

<u>Question pour le rapport :</u> Quelles sont les valeurs typiques du volume dans différentes situations (silence, fond de bruit, on parle au robot) ?

Ne soyez pas déçu si le robot ne vous répond pas…

Les instructions conditionnelles

Dans un programme, il faut pouvoir exécuter des instructions ou pas selon que des conditions soient vrai ou fausse. On les appelle les instructions conditionnelles car elles permettent de faire des choix. Le bloc suivant

permet de faire un choix selon qu'une condition est vraie ou fausse, c'est-à-dire d'exécuter ou non la séquence d'instruction qui est à l'intérieur.

permet de faire une séquence si la condition est vraie et une autre sinon.

Pour générer les conditions, on utilise des opérateurs booléens qui rendent faux/vrai comme réponse :

égalité

infériorité

supériorité

le ET logique

le OU logique

le complément logique

Les boucles et les conditions sont les structures de contrôle qui se trouvent dans tous les langages de programmation. Ces commandes servent à gérer l'ordre d'exécution d'un logiciel, et elles sont essentielles pour créer des logiciels complexes et efficaces.

Exercice 7 : conditions Dans le programme précédent, n'afficher le volume que quand vous parlez au robot, sinon afficher « silence ».

Le capteur ultrason

Exercice 8 : capteur ultrason Le robot est équipé d'un capteur à ultrasons (ses « yeux ») qui mesure la distance entre celui-ci et le premier obstacle rencontré, il s'agit du principe des sonar utilisés par les chauves-souris. Ce capteur est relié sur un connecteur numéroté. En utilisant le bloc

, afficher toutes les secondes la distance mesurée par le capteur à ultrasons. Vérifiez que l capteur ultrason est bien connecté au port 10 de la carte.

Question pour le rapport : Quelles sont les valeurs minimales et maximales mesurées, et à quelles distances réelles correspondent-elles ?

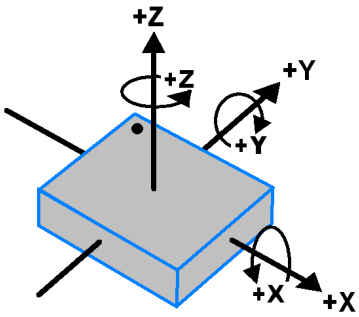
Exercice 9 : ultrason et LEDs Afficher la distance en continu au moyen de l'anneau de LED, distance minimale, 0 LED allumée, distance maximale 12 LED allumées en blanc, autres distances nombre de LED allumées proportionnel à la distance. Utiliser le bloc

afin de créer une valeur qui est certainement entière.

L'accéléromètre / gyroscope

Le robot est équipé d'un capteur permettant de mesurer son orientation en fonction de 3 axes. Le bloc suivant

sera utilisé pour récupérer cette orientation par rapport à l'un des axes (ici l'axe *z*).



MPU-6050
Orientation & Polarity of Rotation

<u>Exercice 10 : gyroscope</u> Afficher les orientations données pour les trois axes en même temps pour en déduire comment est orienté le capteur (sens de *x*, *y* par rapport à l'avant du robot) ainsi que les plages de variation selon *x*, *y* et *z*. Le bloc

peut vous aider à afficher plusieurs variables en même temps.

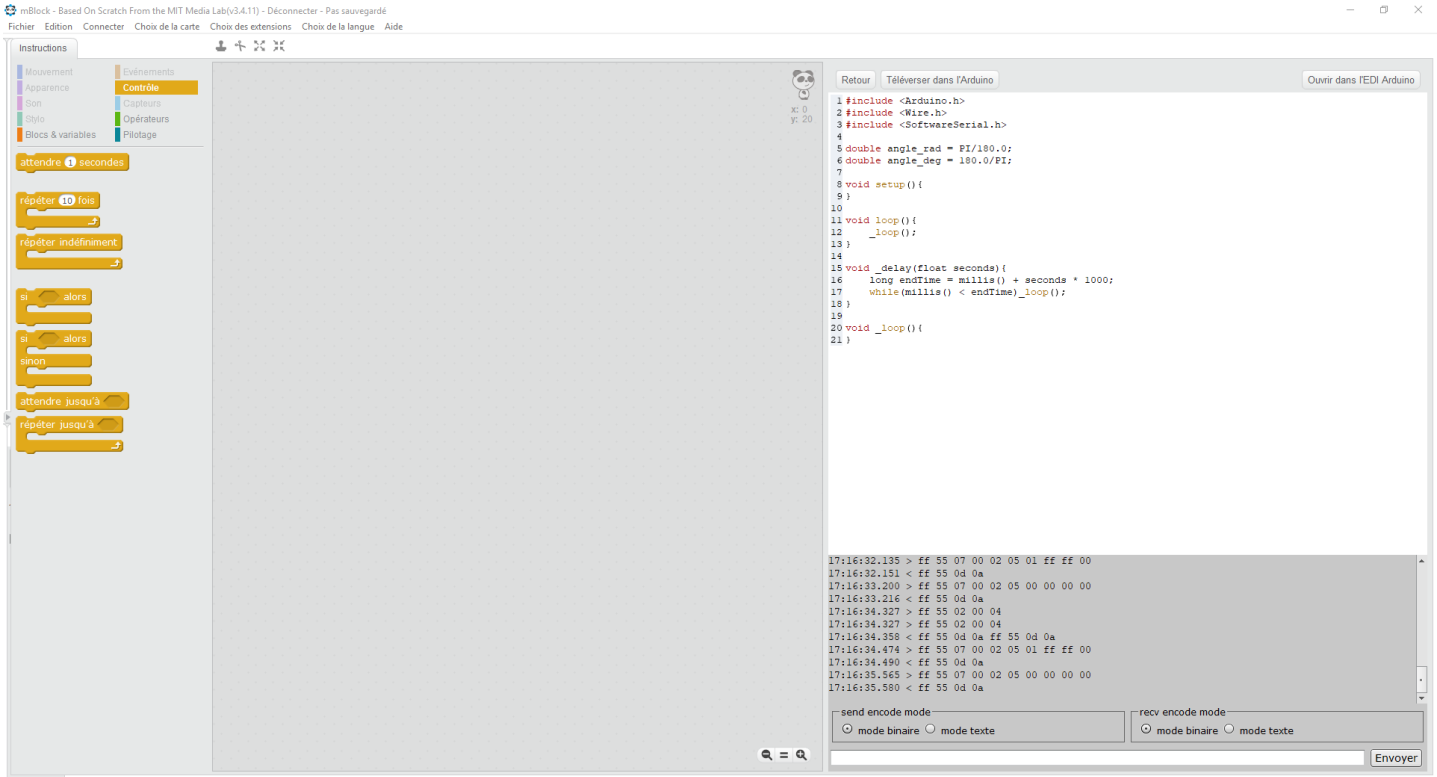
<u>Questions pour le rapport :</u> Pour *x*, *y*, et *z* : Quel est l'axe de rotation ? Quelle est la plage de variation pour chaque axe ? On représentera ceci sur un dessin. Qu'est-ce qui détermine la direction *z* = 0 ?

Les blocs pour la mobilité

Nous allons maintenant étudier comment faire bouger le robot. Pendant la programmation, déconnecter le connecteur d'alimentation à côté du connecteur USB.

CE CONNECTEUR NE SERA REMIS QUE QUAND LE ROBOT EST A TERRE POUR ESSAYER UN PROGRAMME SUCCEPTIBLE DE LE FAIRE BOUGER

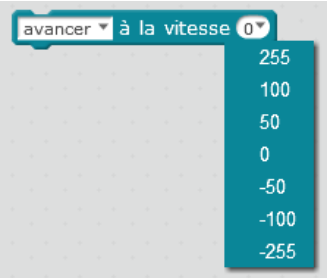
- Passer maintenant en mode Arduino dans le menu ***« Edition »***, l'interface change puisque nous pouvons voir le code généré dans une fenêtre à droite.



- Les programmes doivent maintenant commencer par le bloc

.

- Le téléversement doit se faire à l'aide du Bouton ***« Téléverser dans l'Arduino »***. A la fin du téléversement, le programme se lance automatiquement d'où la précaution de toujours téléverser avec le connecteur débranché. Pour revenir au mode **« non-Arduino »** utilisé auparavant, décocher ***« Mode Arduino »*** dans le menu ***« Edition »*** et réinitialiser le programme par défaut (menu ***« Connecter »***).
- Une fois le programme est téléversé, débrancher le câble USB, mettre le robot à terre et brancher le câble d'alimentation. Appuyer sur le bouton rouge à côté de l'alimentation pour allumer. Pour terminer le programme, appuyer sur le bouton marche-arrêt au-dessus du boitier ou sur le bouton rouge. Pour relancer, appuyer encore sue un des deux boutons.
- Le bloc qui permet d'avancer dans les deux sens est le suivant :



Selon la vitesse désirée, on met un nombre de valeur absolue entre 0 et 255, positif pour une marche avant et négatif pour une marche arrière. Le premier argument permet de choisir si le robot avance, recule, ou tourne.

Exercice 11 : avancer une seconde Faire un programme qui fait avancer en marche avant le robot à la vitesse 128 pendant une seconde.

Exercice 12 : avancer 30 cm Faire un logiciel qui fait avancer le robot par 30 cm à une vitesse de 150 à l'aide du bloc

.

Questions pour le rapport : Quelle est la différence entre les deux blocs

et

? En particulier, quand est-ce que le robot s'arrête dans chaque cas ? Quelle est la signification du nombre de degrés dans le deuxième bloc, c.a.d. qu'est-ce qui tourne par X° ? En particulier, calculez (ne pas mesurer !) la distance par laquelle le robot avance pour 360°.

Exercice 13 : arrêt avant obstacle Faire un programme qui fait avancer le robot à une vitesse de 128 et qui l'arrête définitivement dès qu'un obstacle est détecté en dessous de 50 centimètres.

Exercice 14 : tourner en direction, v1 Créer une variable ***a***. Tourner le robot à droite jusqu'à ce que son orientation (axe *z* du gyroscope) atteigne la valeur donnée dans la variable ***a***. Tester votre programme avec différentes valeurs de la variable ***a***, en particulier ***a*** = 90°, 150°, 270°, et 360° (un tour complet). Il est obligatoire d'utiliser le bloc

en mode *« tourner à droite »* pour cet exercice.

Exercice 15 : tourner en direction, v2 Créer une variable ***a***. Tourner le robot à droite jusqu'à ce que son orientation (axe *z* du gyroscope) atteigne la valeur donnée dans la variable ***a***. Tester votre programme avec différentes valeurs de la variable ***a***, en particulier ***a*** = 90°, 150°, 270°, et 360° (un tour complet). Il est obligatoire d'utiliser le bloc

en mode *« tourner à droite »*, et le gyroscope pour déterminer quand le robot s'arrête.

Questions pour le rapport : Discutez les avantages et inconvénients des deux approches au même problème en exercices 14 et 15.

Exercice 16 : tourner par angle Créer une variable ***a*** qui détermine l'angle de rotation. Tourner le robot à droite par cet angle ***a*** trois fois de suite. Après chaque rotation par ***a***, arrêter le robot et allumer les LEDs pendant 1 seconde. Tester le logiciel d'abord avec ***a*** = 90°, puis avec ***a*** = 150°. En vue des exercices suivants, implémentez une solution qui utilise le gyroscope pour déterminer l'orientation du robot.

Exercice 17 : distance vs orientation Créer une variable ***a***. Faire un logiciel qui tourne le robot par 360° autour de lui-même avec des pas de ***a*** : tester avec ***a*** = 10° et ***a*** = 15°. Pour chaque orientation, mesurer la distance avec le capteur ultrason et afficher la distance mesurée à l'aide des LEDs (voir Exercice 9 : ultrason et LEDs). Utiliser une intensité très basse pour les LEDs afin d'économiser de l'énergie car le moteur est très énergivore.

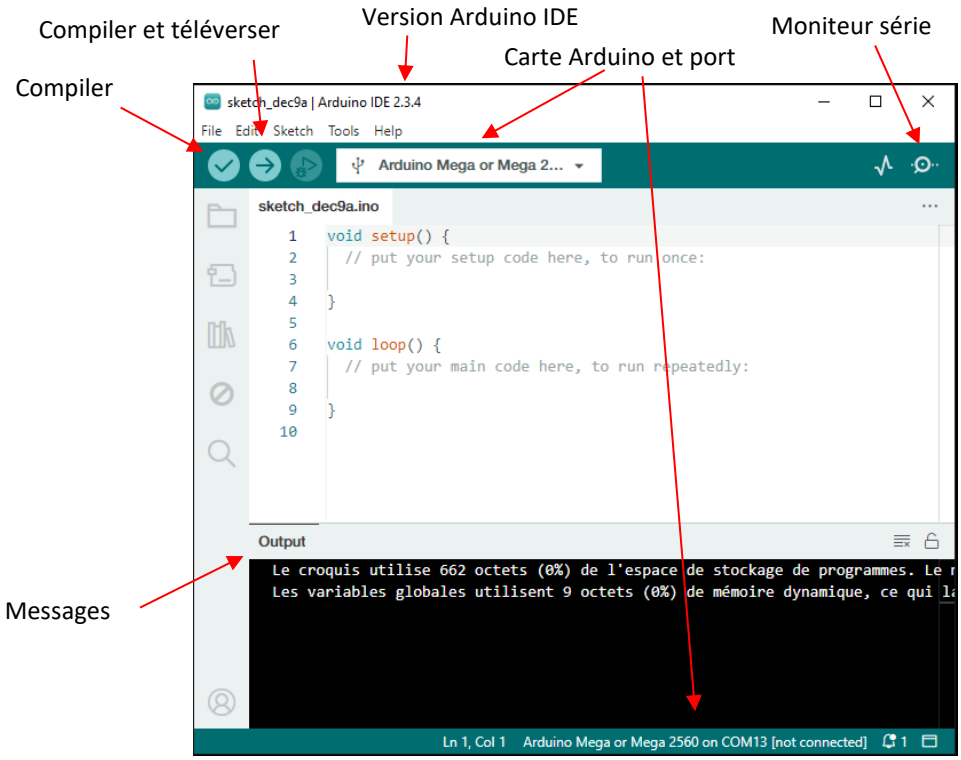
Maintenant vous allez combiner les éléments des exercices précédents pour un programme complexe.

Exercice 18 : contourner un obstacle Faire un programme qui fait rouler le robot ave une vitesse 128 et qui arrête provisoirement le robot dès qu'un obstacle se trouve à une distance inférieure à 30 centimètres. Aussitôt après la période d'arrêt, le robot tourne sur lui-même à 360° pour détecter la plus grande zone libre d'obstacle (obstacle le plus loin). Il se dirigera ensuite vers la première zone la plus dégagée qu'il a détecté.

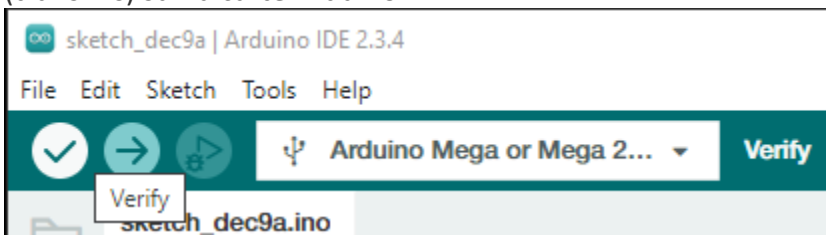
Programmation avancée sous l'environnement Arduino

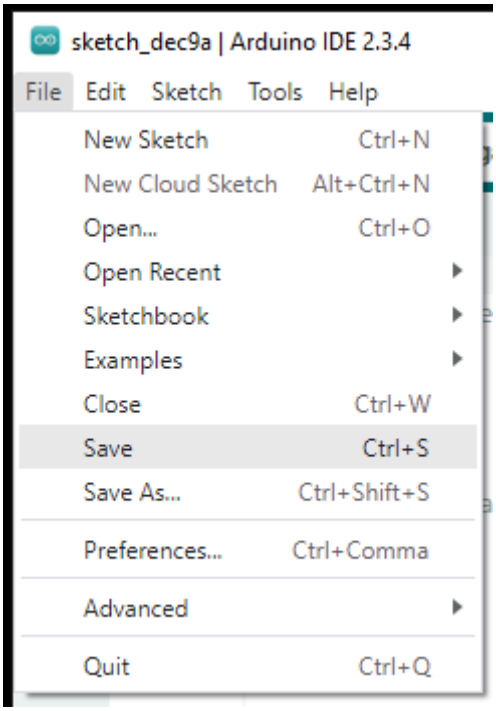
L'environnement développement Arduino

L'IDE Arduino s'utilise sur l'ordinateur pour créer, ouvrir et changer les croquis ou sketch en anglais (Arduino appelle les croquis ou sketch des programmes). Les croquis définissent ce que votre carte Arduino fera. Vous pouvez soit utiliser les boutons situés en haut de l'IDE ou les liens du menu plus haut.



Vérifier : Avant que votre programme "code" puisse être envoyé à la carte, il a besoin d'être vérifié afin de s'assurer qu'il ne comporte pas d'erreur avant d'être téléversé (transmis) sur la carte Arduino.



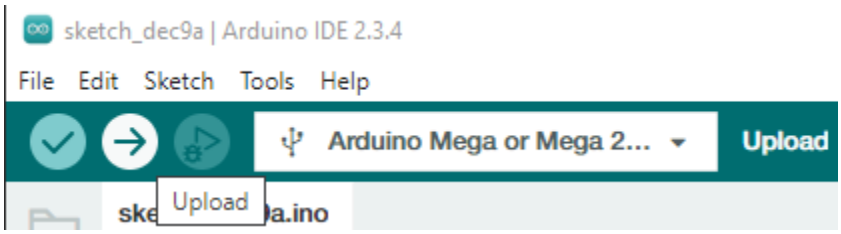


New Sketch / Nouveau : Cela ouvre une nouvelle fenêtre pour créer le nouveau croquis ou sketch (programme)

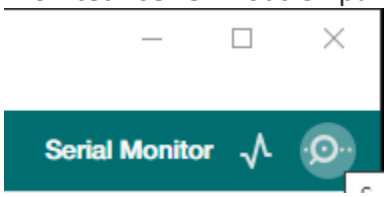
Open / Ouvrir : Cela charge un croquis (sketch ou programme) à partir d'un fichier (extension .ino) sur votre ordinateur.

Save / Enregistrer : Ceci sauve les changements du croquis sur lequel vous travaillez.

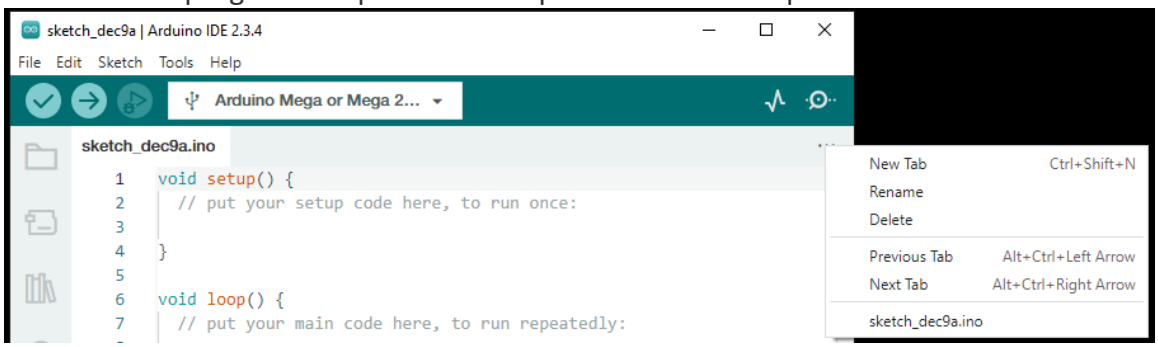
Upload / Téléverser : Cela compile et transmet le croquis via le câble USB à votre carte Arduino.



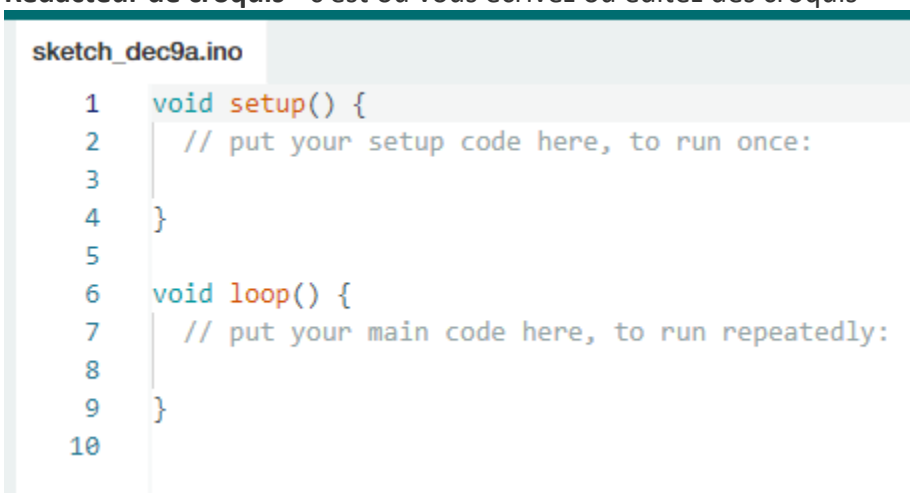
Moniteur série : Nous en parlerons plus tard



Touche à onglet : Cela vous permet de créer plusieurs fichiers dans votre croquis. Cela concerne des programmes plus avancés que nous ne ferons pas ici.



Rédacteur de croquis - c'est où vous écrivez ou éditez des croquis



Console de texte : Elle vous montre ce que l'IDE fait actuellement et elle indique également les messages d'erreur si le logiciel contient une erreur typique de programme (souvent appelée erreur de syntaxe).

Numéro de ligne : cela vous indique le numéro de ligne et de colonne sur lequel votre curseur est activé. C'est utile puisque le compilateur donne des messages d'erreur avec un numéro de ligne.

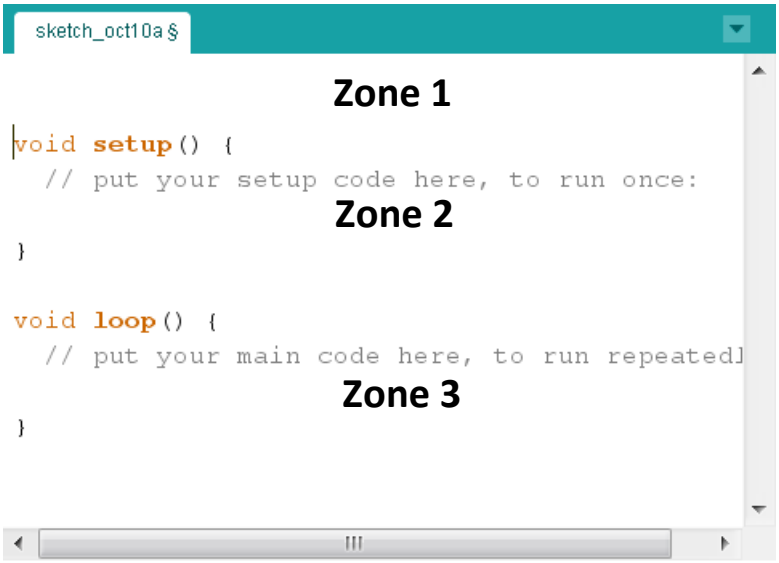
Structure d'un programme Arduino

Un programme Arduino est constitué de 3 zones :

La zone 1 (avant void setup) qui permet de préciser les librairies (programmes déjà fait et réutilisables) et de déclarer et initialiser des variables.

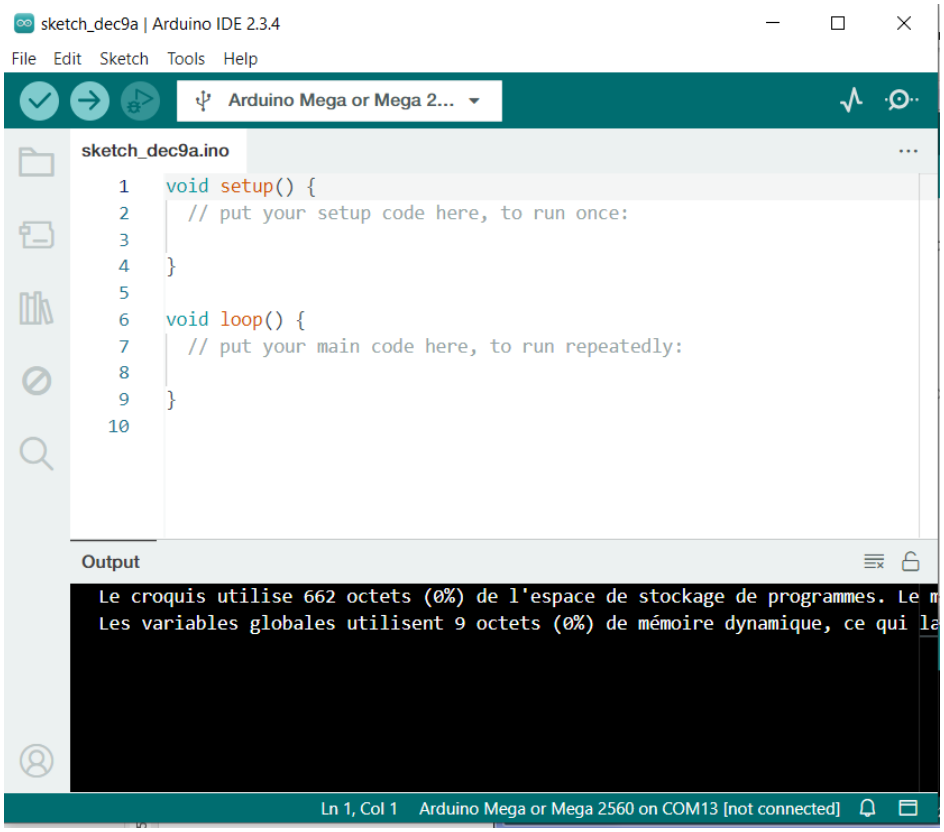
La zone 2 (void setup) qui permet d'initialiser des variables, de paramétrer certaines fonctionnalités (port en sortie ou en entrée), … Le code dans cette zone est exécuté une fois au début.

La zone 3 (void loop) qui est une boucle infinie dans laquelle on placera du code qui sera répété tant que le programme fonctionnera.



Connexion avec la carte

Lancer Arduino, vous aurez la fenêtre suivante qui apparaitra :



- Connectez le robot,
- Dans ***« Outils »***, ***« Type de carte »***, choisir la carte « **Arduino Mega or Mega 2560 »**,
- Dans ***« Outils »***, ***« Ports »***, choisir le bon numéro de port COM (souvent le plus élevé)

Première connexion et installation des pilotes :

Lors de la première utilisation de l'IDE Arduino, il manque souvent les pilotes. Voir l'annexe « installation des pilotes » pour les installer.

Eléments de langage

La programmation Arduino se fait en langage C et nous disposons donc de toutes les possibilités de ce langage pour faire des boucles, des instructions conditionnelles, déclarer des variables, …

Vous trouvez plus de détails sur le site d'Arduino, y inclus une liste de commandes de base qui peuvent vous être utiles : lien

Boucle simple

Une boucle qui va faire varier une variable i d'une valeur v1 à une valeur v2 incluse par pas de 1 s'écrit :

```
for (i=v1; i<=v2; i++)
{
< code à répéter >
}
```

Entre les accolades ouverte et fermé, on place le code à répéter.

On peut également écrire ceci ainsi :

```
i=v1;
do
{
  i++;
}while (I <= v2);
```

ou

```
i = v1;
while(I <= v2)
{
  Serial.println(i);
  i++;
}
```

Moniteur série

Le dernier programme permet d'afficher la valeur de i dans la fenêtre du moniteur série. En général, le moniteur série permet d'afficher des messages venant de la carte, ce qui est utile en particulier pour trouver les erreurs dans un programme.

Il faudra au préalable initialiser le port série dans le setup() au moyen de la ligne Serial.begin(9600); L'argument est le « baud rate », c.a.d. la vitesse de transmission en bit par seconde. Il est toujours 9600.

Après cette initialisation, on peut afficher dans le moniteur série la valeur d'une variable avec la ligne Serial.println(i); ou un message texte avec Serial.println('texte');

Instructions conditionnelles

Les branchements conditionnels s'effectuent avec les instructions if et éventuellement else ou else if.

Exemples :

```
i = 5;
if (i==5)
{
  Serial.println("i est egal a 5");
}
```

Ou

```
i = 5;
if (i==5)
{
  Serial.println("i est egal a 5");
}
else
{
  Serial.println("i n'est pas egal a 5");
}
```

Ou

```
i = 5;
if (i==5)
{
  Serial.println("i est egal a 5");
}
else if (i==10)
{
  Serial.println("i est egal a 10");
}
else
{
  Serial.println("i est ni egal a 5, ni égal à 10");
}
```

Les accolades ne sont pas obligatoires quand il n'y a qu'une seule instruction après un if, un else ou un else if.

Opérateurs booléens et comparaisons

ET logique:	&&	(AND)
OU logique:	||	(OR)
Complément logique:	!	(NOT)

Supérieur : >
Supérieur ou égal : >=
Inférieur : <
Inférieur ou égal : <=
Egal : ==

Attention, == est une comparaison qui rend true si les expressions à gauche et à droite donnent la même valeur. Par exemple, 4 == 5 rend false.
Par contre = attribue la valeur de l'expression à droite à la variable qui est à gauche. Par exemple, a = 4 +5 met la variable a à 9.

Syntaxe générale

- Toutes commandes (à l'exception de certaines comme if, while, for) se termine avec un ;
- Après // on met les commentaires qui sont ignorées lors de l'exécution
- Le code dans une fonction [comme void setup() et void loop ()] ou certaines commandes (if, while, for) est entouré par des accolades {<code>}.

Variables

En langage C, on peut définir des variables de type entier, décimal simple précision, décimal de type double précision, des caractères, des variables logiques, …

Exemple :

```
int i=8; // entier normal 2 octets
boolean x=true; //variable logique true ou false
float y=3.14; //Nombre décimal simple précision
double z=3.145686; //Idem float mais double précision
unsigned int u=255; //variable entière non signée 2 octets
short s=-78; //entier court 2 octets
long l=14586254; //entier long 4 octets
char c='F'; //contient un code ASCII 1 octet
byte b=127; //entier sur un octet
```

Avant d'utiliser une variable, on doit la déclarer : p.ex. la ligne int i; déclare une variable du type int dont le nom est i. Les variables déclarées dans la zone 1 sont accessibles dans tout le logiciel ; en revanche une variable déclarée dans la partie void setup() [zone 2] ou void loop() [zone 3] est seulement accessible dans cette partie. Plus précisément, une variable déclarée entre deux accolades { … } est accessible seulement dans cette partie.
On utilise p.ex. i = 5; pour attribuer une valeur à une variable. La déclaration d'une variable ne lui attribue pas une valeur spécifique ! Mais on peut écrire p.ex. int i=5; pour déclarer la variable i et lui attribuer la valeur 5 en même temps. Il est recommandé de toujours donner une valeur bien définie à toutes les variables dès leur déclaration afin d'éviter des erreurs dues à une valeur aléatoire de la variable.

Exercice 19 : taille de variables La fonction sizeof() permet d'afficher le nombre d'octets qu'occupent un type de variable [par exemple sizeof(long)]. Ecrire un programme qui affiche sur le moniteur la taille occupée par chacun des types de variables utilisés dans l'exemple précédent.

Question pour le rapport : Donner la taille pour chaque type de variable mentionné ci-dessus.

Exercice 20 : exécution unique Reprendre le code de l'exercice précédent et le mettre dans void loop() s'il n'y est pas encore. Trouver une astuce afin que la boucle ne s'exécute qu'une fois malgré la boucle principale loop() du programme principal.

Exercice 21 : i=0…100 Faire varier et afficher dans le moniteur de série une variable i entre les valeurs 0 et 100.

Mon premier programme avec le robot

- Au-dessus du setup(), *(zone 1)* inclure les bibliothèques essentielles du robot en ajoutant :
  ```
  #include "MeAuriga.h"
  #include "Wire.h"
  ```
- Déclarer ensuite un objet led que l'on initialisera en précisant le numéro de la première LED (0) et en précisant le nombre de LED (12)
  ```
  MeRGBLed led(0,12);
  ```
- On précise ensuite dans le setup() *(zone 2)* que l'anneau de LEDs est relié à la broche digitale n° 44 *(vérifiez ceci sur le schéma de la carte Auriga)*
  ```
  led.setpin(44);
  ```
- Allumons dans la boucle la 5ème LED avec une couleur bleue,
  ```
  led.setColor(5, 0, 0, 255);
  led.show();
  ```
- Téléversez le programme et constatez l'allumage de la LED dans la couleur programmée.

Exercice 22 : guirlande lumineuse Faire un programme qui allume chaque LED une par une pendant 200 millisecondes, l'éteint puis allume la suivante, … Après d'avoir allumé la douzième LED, on revient à la première et on un reprend indéfiniment le cycle. Utiliser la commande delay(x) pour pauser l'exécution du logiciel pendant x millisecondes.

Bonus 1 : résoudre cet exercice sans d'utiliser une boucle (for, while, do)

Bonsu 2 : résoudre cet exercice en utilisant ni boucle, ni condition (if)

Les capteurs de lumière

La carte Auriga est équipée avec deux capteurs de lumière. Prendre le programme mBlock écrit précédemment pour mesurer dans votre éclairage actuel les valeurs min lmin *(main sur le boitier)* et max lmax *(éclairement maximal sur votre table)*. Mémorisez ces valeurs. Pour définir un objet capteur de lumière, il faut utiliser la classe MeLightSensor et définir une instance de cette classe (veuillez noter la terminologie

orienté objet) en précisant le port du capteur. Sur la carte Auriga, on utilise le port 12 pour le capteur 1 et le port 11 pour le capteur2 :

```
MeLightSensor capteurL1(12);
MeLightSensor capteurL2(11);
```

La méthode read() retourne la valeur de luminosité actuelle, capteurL1.read(); p.ex. donne la valeur du capteur 1.

Exercice 23 : afficher luminosité par LEDs Ecrire un programme qui allume successivement les LED de l'anneau en fonction de l'éclairage du capteur 1. A la valeur Lmin, aucune LED est allumée, à la valeur Lmax, toutes les LED sont allumées, entre les deux valeurs on allumera proportionnellement les LED en fonction de la mesure.

Exercice 24 : comparaison d'éclairage Ecrire un programme qui allume les 6 premières LEDs si le capteur 1 reçoit plus de lumière que le capteur 2 et les 6 autres LEDs sinon. En cas d'égalité, aucune LED n'est allumée.

Le capteur à ultrasons

Le robot présente un capteur à ultrasons capable de détecter un obstacle de 3 à 400 cm. Les fonctions contenues dans la bibliothèque Makeblock sont :

MeUltrasonicSensor capteurUS(PORT_X) où PORT_X est le port 6 ou 7 ou 8 ou 9 ou 10 sur Auriga sur lequel peut être branché le module (normalement il s'agit du port 10).

A partir de l'objet capteurUS, on peut appeler la méthode distanceCm() pour reçupérer la distance en centimètre entre le capteur et l'obstacle le plus proche, c.a.d. capteurUS.distanceCm() renvoie la distance en cm.

Exercice 25 : afficher distance Faire un programme qui affiche en permanence la distance mesurée sur le moniteur série.

Exercice 26 : afficher distance par LEDs Refaire le même exercice que dans l'environnement mBlock (voir Exercice 9 : ultrason et LEDs) en allumant progressivement les LED en fonction de la distance.

Le capteur de son

Pour utiliser le capteur de son embarqué, il faut utiliser la classe MeSoundSensor en déclarant un objet son de cette façon :

```
MeSoundSensor son(14);
```

A partir de là, la méthode strength() permettra de retourner la mesure du bruit dans votre environnement.

Exercice 27 : afficher bruit Faire un programme qui affiche en permanence la valeur du bruit dans le moniteur série. Observez et notez la valeur du bruit ambiant et essayez de faire un bruit en claquant des doigts ou les mains. Il est possible que vous n'ayez pas le temps de voir la valeur ponctuellement haute dans le moniteur. Ajoutez une condition dans votre code qui bloque l'affichage pendant 2 secondes [delay(2000)] si le

bruit mesuré est sensiblement supérieur à celui ambiant. Notez la valeur du bruit que vous avez généré, elle servira par la suite.

Exercice 28 : afficher distance en fonction du bruit Faire un programme qui déclenche et stoppe la mesure et l'affichage de la distance avec le capteur à ultrasons par un bruit que vous générerez. Coordonnez-vous pour tester vos programmes car les bruits des uns vont interférer sur le bruit des autres.

Le gyroscope

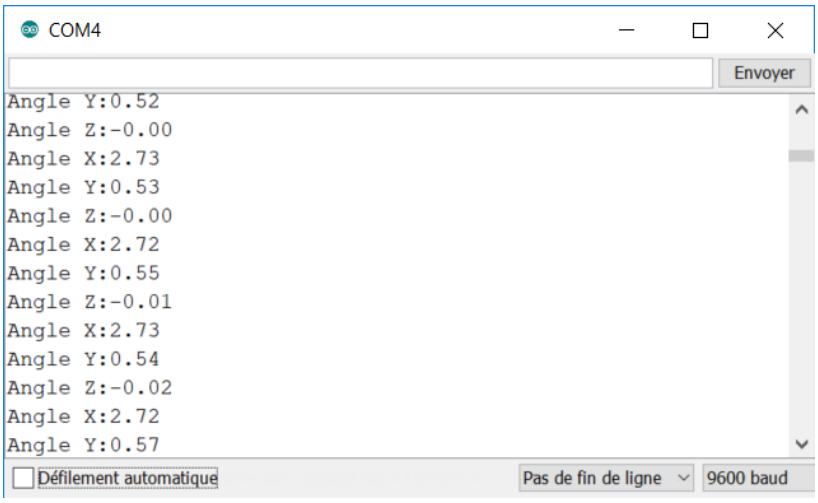
Pour lire les valeurs du gyroscope, il faut utiliser la classe MeGyro pour initialiser par exemple un objet gyro en précisant le port 0 et l'adresse I2C du composant (mettre 0x69), 0x signifiant que l'on exprime l'adresse en hexadécimale.

```
MeGyro gyro(0, 0x69);
```

On utilisera ensuite la méthode begin() dans le setup() pour initialiser le gyroscope ainsi que la méthode update() avant chaque acquisition pour calculer les angles. On utilisera juste ensuite les méthodes getAngleX(), getAngleY() et getAngleZ() pour récupérer les angles *x*, *y*, et *z*.

Vérifiez que l'on a les mêmes variations que dans Mblock (voir Exercice 10 : gyroscope) !

Exercice 29 : afficher orientation Lire et afficher en continue les différents angles sur le moniteur comme sur la copie d'écran ci-dessous :



Pilotage du robot

Le robot pilote deux moteurs à courant continu à travers deux slots intégrés à la carte. Pour accéder aux fonctionnalités liées au moteur, il faut utiliser la classe MeEncoderOnBoard et déclarer un objet de cette classe pour chacun des deux moteurs en précisant le slot concerné.

```
MeEncoderOnBoard Moteur1(SLOT1);
```

```
MeEncoderOnBoard Moteur2(SLOT2);
```

☝ Mettre le robot à terre ou sur un support laissant les roues tourner dans le vide et brancher le connecteur alimentant les moteurs à partir des piles.

Pour faire tourner un moteur, on utilisera la méthode setMotorPwm(vel), où vel est une variable entre -255 et +255 qui représente la vitesse. 50 correspond à une vitesse assez faible. Si le robot ne bouge pas, augmenter cette valeur légèrement. Recommencez jusqu'à avoir un début de mouvement dans le même sens pour les deux chenilles. *(Attention, il faut donc que les moteurs tournent dans des sens opposés et une des deux vitesses doit être négative)*.

Faire ceci pour les deux moteurs et limiter leur fonctionnement à 2 secondes au moyen du programme. Voici le programme de la boucle (attention, n'oubliez pas de déclarer vel dans la zone 1 du programme …)

```
void loop() {
  Moteur1.setMotorPwm(-vel);
  Moteur2.setMotorPwm(vel);

  delay (2000);

  vel = 0;
  Moteur1.setMotorPwm(vel);
  Moteur2.setMotorPwm(vel);
}
```

Si vous voulez arrêter le programme, appuyer sur le bouton rouge au-dessus du capot de la plaque ou débrancher les piles.

Exercice 30 : tourner par 360° Faire un programme qui fait tourner le robot sur lui-même dans le sens des aiguilles d'une montre sur 360° par rapport à sa position initiale.

Exercice 31 : mesurer vitesse Faire un programme qui mesure la vitesse d'un objet et allume les LEDs (en très basse intensité) en fonction de la valeur mesurée ; optimiser le délai de mesure.

Exercice 32 : suivre à distance Faire un programme qui fait le robot vous suivre à une certaine distance. La distance, paramètre d'entrée, doit rester constante en avançant ou en reculant.

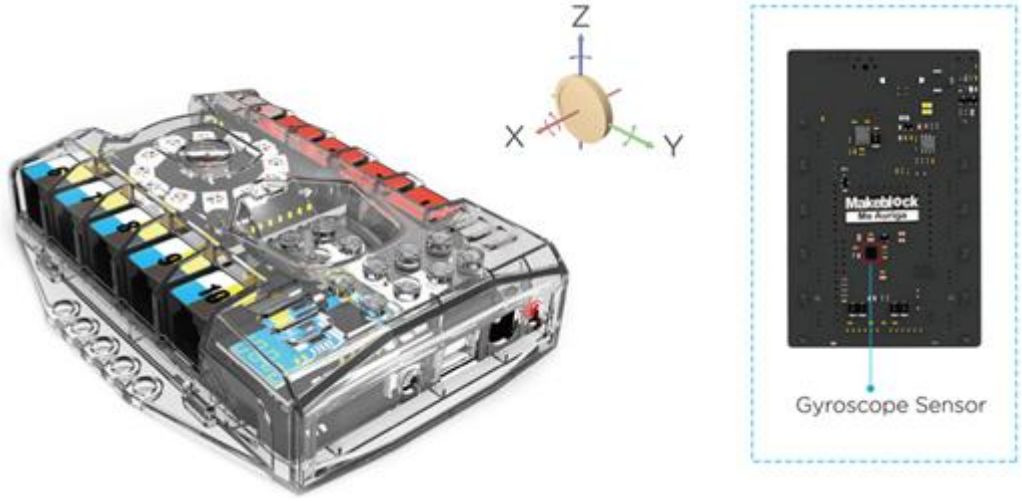
Exercice 33 : virage Faire un programme qui fait toujours avancer le robot (pas à grande vitesse), le robot doit adapter son virage (vitesse entre les 2 chenilles) en fonction de la distance de l'obstacle. Plus la distance est courte, plus le virage est serré.

Exercice 34 : déplacement autonome Faire un programme qui fait le robot se déplacer au mieux dans la salle de TP : recul, gestion des obstacles, adaptation de la vitesse etc. La vitesse pourra être variable en fonction de la distance mesurée. L'anneau

de LEDs devra indiquer des informations sur l'environnement ou le comportement du robot en fonction de ce qu'il fait.

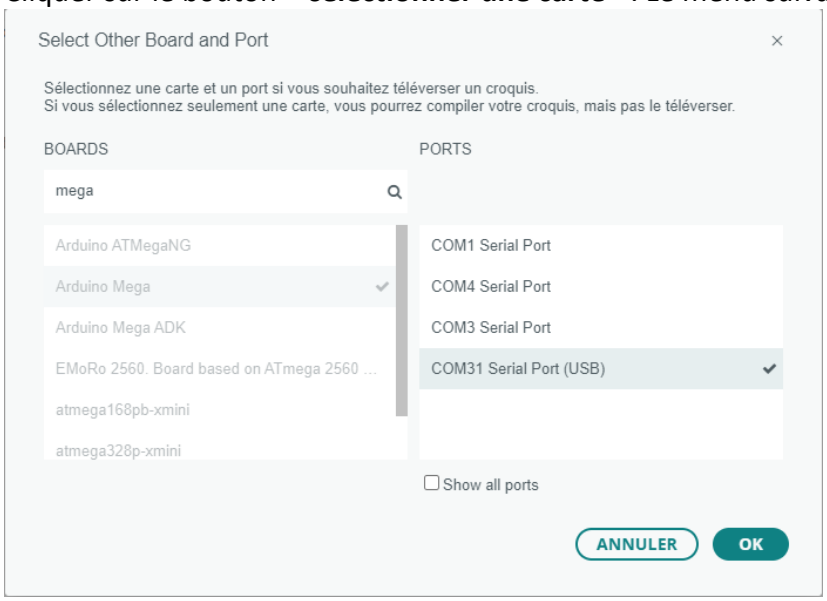
Annexes

Gyroscope



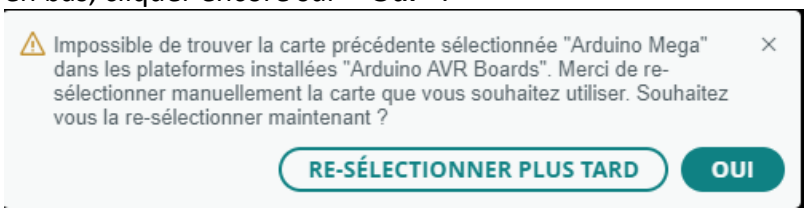
IDE Arduino : installation des pilotes

- Cliquer sur le bouton **« sélectionner une carte »**. Le menu suivant s'ouvre :



- Choisir le bon port COM
- Chercher **« mega »**, puis sélectionner **« Arduino Mega »** dans la liste en-dessous.
- Cliquer sur **« OK »** pour fermer le menu.
- En bas à droite un dialogue s'affiche qui demande d'installer les pilotes manquants. Cliquer sur **« Oui »**.

- Attendre l'installation, puis cliquer encore sur « Oui ». Un autre dialogue s'ouvre en bas, cliquer encore sur **« Oui »**.



- Le menu de selection de carte se rouvre. Maintenant vous pouvez choisir « **Arduino Mega or Mega 2560** ».

Liste des exercices

Exercice 1 : capteur de luminosité 19
Exercice 2 : boucle infinie 20
Exercice 3 : boucle N fois 20
Exercice 4 : variable 20
Exercice 5 : LEDs 20
Exercice 6 : capteur de son 21
Exercice 7 : conditions 22
Exercice 8 : capteur ultrason 22
Exercice 9 : ultrason et LEDs 22
Exercice 10 : gyroscope 23
Exercice 11 : avancer une seconde 25
Exercice 12 : avancer 30 cm 25
Exercice 13 : arrêt avant obstacle 25
Exercice 14 : tourner en direction, v1 25
Exercice 15 : tourner en direction, v2 25
Exercice 16 : tourner par angle 25
Exercice 17 : distance vs orientation 25
Exercice 18 : contourner un obstacle 26
Exercice 19 : taille de variables 35
Exercice 20 : exécution unique 35
Exercice 21 : i=0…100 35
Exercice 22 : guirlande lumineuse 35
Exercice 23 : afficher luminosité par LEDs 36
Exercice 24 : comparaison d'éclairage 36
Exercice 25 : afficher distance 36

Exercice 26 : afficher distance par LEDs 36

Exercice 27 : afficher bruit 36

Exercice 28 : afficher distance en fonction du bruit 37

Exercice 29 : afficher orientation 37

Exercice 30 : tourner par 360° 38

Exercice 31 : mesurer vitesse 38

Exercice 32 : suivre à distance 38

Exercice 33 : virage 38

Exercice 34 : déplacement autonome 38